```
module DSP(A, B, C, D, clk, rst_n, P);
parameter OPERATION = "ADD";
input  [17:0] A, B, D;
input  [47:0] C;
input clk, rst_n;
output [47:0] P;

// Stage 1 registers
reg [17:0] S1_A,S1_B,S1_D;
reg [47:0] S1_C;

// Stage 2 registers
reg [18:0] S2_A, S2_add;
reg [47:0] S2_C;

// stage 3 registers
reg [36:0] S3_mul;
reg [47:0] S3_C;

// stage 4 registers
reg [47:0] S4_P;


// stage1
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        S1_A <= 0;
        S1_B <= 0;
        S1_C <= 0;
        S1_D <= 0;
    end
    else begin
        S1_A <= A;
        S1_B <= B;
        S1_C <= C;
        S1_D <= D;
    end
end

```

```
// stage2
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        S2_A <= 0;
        S2_add <= 0;
        S2_C <= 0;
    end
    else begin
        S2_A <= S1_A;
        if (OPERATION == "ADD") begin
            S2_add <= S1_B + S1_D;
        end
        else if (OPERATION == "SUBTRACT") begin
            S2_add <= S1_D - S1_B;
        end
        S2_C <= S1_C;
    end
end

// stage3
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        S3_mul <= 0;
        S3_C <= 0;
    end
    else begin
        S3_mul <= S2_A * S2_add;
        S3_C <= S2_C;
    end
end
```

```
// stage4
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        S4_P <= 0;
    end
    else begin
        if (OPERATION == "ADD") begin
            S4_P <= S3_C + S3_mul;
        end
        else if (OPERATION == "SUBTRACT") begin
        S4_P <= S3_C - S3_mul;
        end
    end
end

assign P = S4_P;
endmodule
```

```
module DSP_tb;

// declare local wire and reg
reg clk_tb, rst_tb;
reg [17:0] A_tb, B_tb, D_tb;
reg [47:0] C_tb;
wire [47:0] P_tb;
integer k;
// Instantiate the module under test
DSP uut(
    A_tb,
    B_tb,
    C_tb,
    D_tb,
    clk_tb,
    rst_tb,
    P_tb
);

// General stimuli

always #5 clk_tb = ~clk_tb;

task assert_reset;
    rst_tb = 1'b0;
    @(negedge clk_tb);
    if (P_tb == 48'b0) begin
        $display("Reset successful");
    end
    rst_tb = 1'b1;
endtask

function [47:0] golden_model;
    input [17:0] A, B, D;
    input [47:0] C;
    reg [17:0] add_result;
    reg [36:0] mult_result;
    begin
        add_result = B + D;
        mult_result = A * add_result;
        golden_model = mult_result + C;
    end
endfunction
```

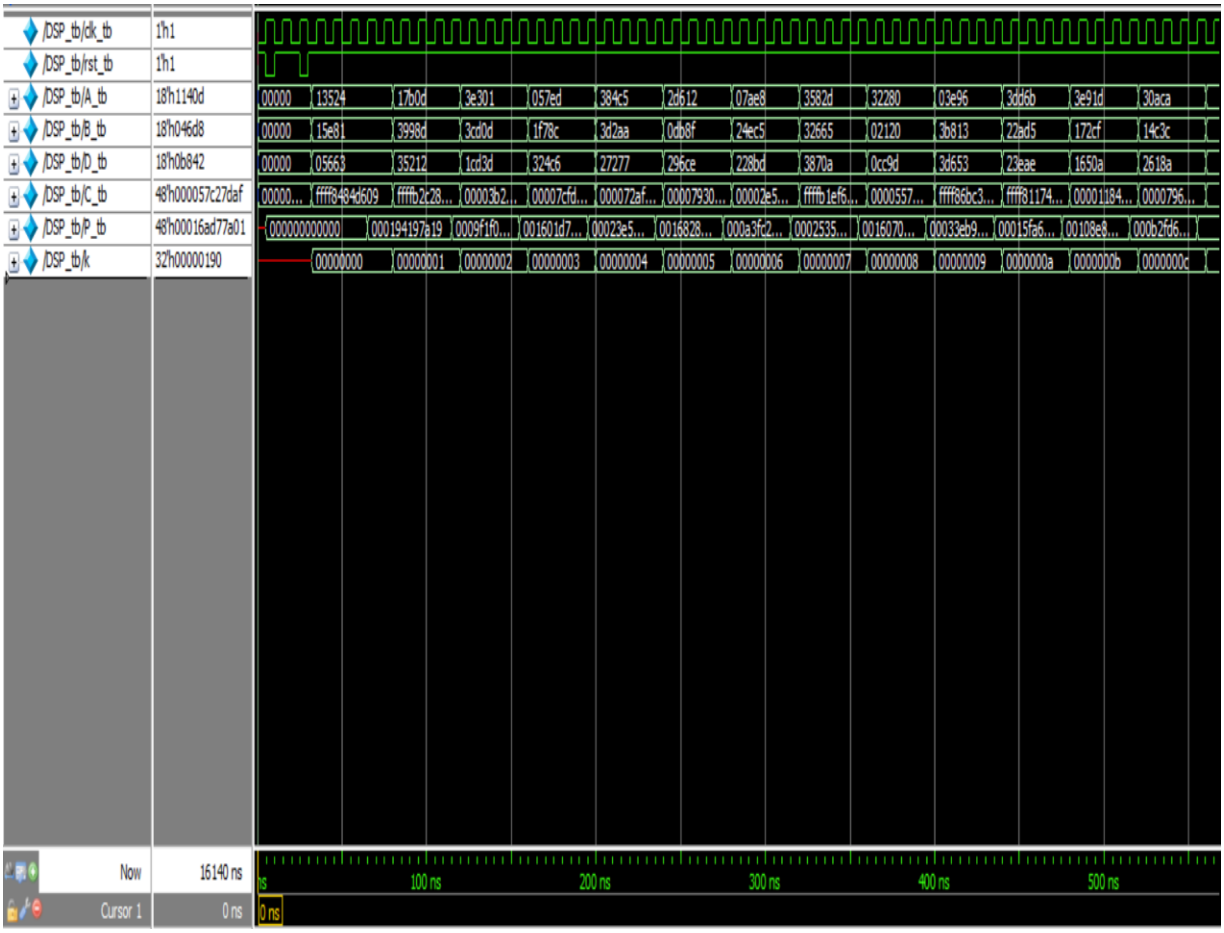
```
task check_result();
    reg [47:0] expected_result;
    expected_result = golden_model(A_tb, B_tb, D_tb, C_tb);
    repeat(4) @(negedge clk_tb);
    if (P_tb !== expected_result) begin
        $display("Test failed: A = %d, B = %d, C = %d, D = %d, Expected P = %d, Actual P = %d", A_tb, B_tb, C_tb, D_tb, expected_result, P_tb);
    end else begin
        $display("Test passed: A = %d, B = %d, C = %d, D = %d, P = %d", A_tb, B_tb, C_tb, D_tb, P_tb);
    end
endtask

initial begin
    clk_tb = 1'b0;
    rst_tb = 1'b1;
    A_tb = 0;
    B_tb = 0;
    C_tb = 0;
    D_tb = 0;

    #5;
    assert_reset;
    #15;
    assert_reset;
    #2;
    for(k = 0; k < 400; k = k + 1) begin
        A_tb = $random;
        B_tb = $random;
        C_tb = $random;
        D_tb = $random;
        #10; // wait for the result to stabilize
        check_result();
    end

    #100;
    $stop;
end


endmodule
```


/DSP_tb/clk_tb
1'h1
/DSP_tb/rst_tb
1'h1
/DSP_tb/A_tb
18'h1140d
00000 13524 17b0d 3e301 057ed 384c5 2d612 07ae8 3582d 32280 03e96 3dd6b 3e91d 30aca
/DSP_tb/B_tb
18'h046d8
00000 15e81 3998d 3cd0d 1f78c 3d2aa 0db8f 24ec5 32665 02120 3b813 22ad5 172cf 14c3c
/DSP_tb/D_tb
18'h0b842
00000 05663 35212 1cd3d 324c6 27277 296ce 228bd 3870a 0cc9d 3d653 23eae 1650a 2618a
/DSP_tb/C_tb
48'h000057c27daf
00000... ffff8484d609 ffffb2c28... 000036b2... 00007cfd... 000072af... 00007930... 00002e5... ffffb1ef6... 0000557... ffff86bc3... ffff81174... 00001184... 0000796...
/DSP_tb/P_tb
48'h00016ad77a01
000000000000 000194197a19 0009f1f0... 001601d7... 00023e5... 0016828... 000a3fc2... 0002535... 0016070... 00033eb9... 00015fa6... 00108e8... 000b2fd6...
/DSP_tb/k
32'h00000190
00000000 00000001 00000002 00000003 00000004 00000005 00000006 00000007 00000008 00000009 0000000a 0000000b 0000000c
Now
16140 ns
Cursor 1
0 ns
100 ns
200 ns
300 ns
400 ns
500 ns

```
================================================================================
=== Instance: /DSP_tb/uut
=== Design Unit: work.DSP
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         8         8         0   100.00%
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      24        24         0   100.00%
```

```
 Toggle Coverage:
     Enabled Coverage              Bins      Hits    Misses  Coverage
     ----------------              ----      ----    ------  --------
     Toggles                        908       908         0   100.00%
```